A rigorous pretraining knowledge base: evidence and reproduction

What this knowledge base is for

The unit of knowledge is a **scoped claim with evidence and a decision consequence**. A paper library tells us what can be read. A synthesis explains a mechanism. An experiment record tells us what was actually run. A qualified reproduction tells us which result we can obtain again. These are connected objects, with different acceptance criteria.

This representation follows KernelWiki's schema, inspected at commit `b6b4301f15e8ce6955a56776690643ce5db369e6`. Its separate source/wiki/query layers, evidence fields, reproducibility levels and validators are useful engineering precedents. We inspected that design; we did not independently execute every upstream quality gate or inherit its factual claims about kernels.

The original literature collection, Jiaxuan blog, supplied notes, PDFs, LaTeX, code archives and result snapshots remain. Source cards are an additional access path, never a replacement.

A claim must answer six questions

Question	Required information	A failure this prevents
What is asserted?	One statement with an evidence kind	Treating a proposed recipe as a measured optimum
Where is the evidence?	Source ID, exact section/table/function/row, frozen artifact	Citing an entire repository for an implementation detail
What was held fixed?	Model, data, optimizer, horizon and evaluation scope	Transferring a coefficient between incompatible recipes
What number is compared?	Metric, value, unit, baseline and protocol	Comparing macro BPB to pooled BPB or sequence batch to token batch
What did we check?	Source read, artifact inspected, CPU replay or training reproduction	Calling a downloaded model card a reproduced model
What remains unknown?	Limits, missing inputs, uncertainty and next discriminating experiment	Hiding cloud access gaps behind a runnable-looking command

The claim ledger instantiates this contract. The raw records are machine-readable. A locator is narrower than a source title: "Step Law Table4" supports a configuration claim; it does not prove which width is best for our hardware.

Confidence and reproducibility are separate axes

`source-reported` records an authoritative account. `inferred` records our synthesis or derivation. `experimental` records an unqualified proposal. `verified` requires named document and implementation evidence; it is not a synonym for statistical certainty or a GPU rerun. Reproduction scope remains explicit even when evidence coverage is strong.

A page may contain a runnable CPU audit of an HF table and still have no runnable training reproduction. Likewise a code snippet can expose an optimizer convention without proving its empirical transfer. This is why every page has `reproducibility_scope`, and numerical results bind to individual claim IDs rather than acquiring a confidence label from the page title.

The current workbench has public-record checks and synthetic analytical fixtures. No model was trained. Coverage reports what is indexed, what is claim-linked, which checks were run and what remains unreviewed.

One example, traced end to end

Claim `claim-marin-transfer-d1536` records the reported d1536 final macro BPB0.91844. Follow its source ID to issue9126, then to the saved issue body and completion comment. The record identifies the old ladder comparator, Paloma16-subset metric and90767 completed updates.

Its1.20× compute-equivalent speedup assumes equal throughput. It is not a measured reduction in training hours. The captured final-rung report supplies no confidence interval. The CPU workbench does not reproduce this training run. These limits are part of the claim, not optional caveats to discard when quoting the headline.

Now compare claim `claim-marin-hf-records`. Its116-row count **is** locally replayed from a pinned public Parquet artifact. Checking the table does not promote the training losses inside those rows to independently reproduced outcomes. Evidence is compositional; verification does not spread automatically to neighboring statements.

Updating evidence without rewriting history

1. Preserve the old artifact and its digest. Add a new version or new source card when upstream changes.
2. Identify affected claims using the version registry and reverse-source index.
3. Read the changed implementation or experimental report, including status corrections and limitations.
4. Decide whether the new evidence confirms, restricts, contradicts or supersedes the old scoped claim.
5. Record the new scope and any changed comparisons. Re-run only the relevant CPU or training protocol.
6. Regenerate indices and PDFs; validate internal references and artifact hashes.

A newer Luxical DAG does not invalidate a historical Harrier result. It changes the reproduction inputs. A closed experiment issue does not establish a winner. A new model checkpoint does not make an old task configuration comparable. Preserve these distinctions in the graph.

Acceptance before giving operational advice

A recommendation should state the decision, mechanism, closest empirical evidence, implementation contract, expected measurements and rejection rule. If a mandatory input is missing, return a conditional plan with that input named. For our96-H100 project, measured useful throughput and access to compatible data/scorer artifacts remain decision inputs, not defaults to fill with confidence.

Validation catches missing fields, broken references, stale receipts, illegal confidence promotion, changed original literature and undeclared question gaps. It cannot establish that a scientific argument is correct. Semantic review still requires reading the source and inspecting the experimental controls.